

FULL PRESENTATION SCRIPT — ~30 MINUTES

Galatiq : Invoice Processing

Everything to say, start to finish — the problem, the goals I set, what I understood, what I built, and why. Read it out loud.

Narrative + live demo + design rationale + Q&A

16 September 2026

HOW TO USE THIS SCRIPT

Read the plain lines, follow the boxes

The **plain black lines** are what you say — out loud, slowly. The ... marks are natural pauses; don't rush them. The coloured boxes are stage directions: **DO** (an action), **CLICK** (a click in the app), **POINT** (where to point), **PAUSE** (stop and let it land), and **NOTE** (a reminder for you, don't say it). Each part shows a rough time so you can pace to ~30 minutes. You don't have to read it word for word — once you've run through it twice, the flow will be yours.

NOTE Before you begin: have the dashboard running (`python -m streamlit run app.py`) with the browser open on the landing screen, and your Gemini key already set in `config.py`. Keep this script beside you.

PART 1

Opening — set the room

~1–2 min

PAUSE Look at them, relaxed. Don't touch the screen yet.

“Hi everyone — thanks for the time. ... I'm going to walk you through the project I built for this case, end to end. ... I'll spend a few minutes on the problem itself, because I think that's what makes the whole thing make sense. ... Then I'll tell you what the goals were, and how I read them. ... Then I'll show you what I built, live, and I'll be honest about the decisions and the trade-offs along the way. ... If a question comes up in the middle, jump in — I'd rather it be a conversation than a lecture.”

“Quick note before I dive in — everything you'll see is running locally on my machine. The reasoning part is powered by an LLM, I'm using Google's Gemini, and I've set my key in the config. ... Alright, let me start with the problem.”

PART 2

The problem — go deep

~3–4 min

DO Stay off the screen. Tell this like a story — it's the 'why' for everything else.

“So the company in the case is Acme Corp — a mid-size, PE-backed manufacturer. ... And they have a very boring, very expensive problem: ... they’re losing around **two million dollars a year** just on *processing invoices*. Not on the invoices themselves — on the *handling* of them.”

“Let me paint the picture, because it’s a mess a lot of finance teams will recognise. ... Invoices come in by email, as PDF attachments, in all sorts of formats. ... Some are clean. ... A lot are not — there are typos, there’s scan noise from where something was photocopied, dates are written five different ways, an item is called ‘Widget A’ on one and ‘WidgetA’ on the next. ... And every one of these, today, a person has to open, read, and re-type into a system by hand.”

“And it doesn’t stop at typing. ... They then have to check it against a legacy inventory database — is this item real, do we have the stock. ... Then they have to get a VP to approve it, usually over an email chain. ... And only then does it go to payment, through a banking API.”

PAUSE *Slow down — hit these three numbers deliberately.*

“So what does all that manual work cost them? ... Three things. ... One — a **thirty percent error rate**. Nearly a third of invoices have a mistake somewhere. ... Two — a **five-day** processing delay, which annoys good vendors and holds up the business. ... And three — and this is the scary one — there’s no real guard against **fraud**. ... A fake invoice with urgent language and a wire-transfer request can slip straight through, because a tired person at 5pm just wants to clear their queue.”

“And think about who’s affected. ... The AP clerk is drowning in a queue and gets blamed for the errors. ... The VP is rubber-stamping email approvals they don’t really have time to read. ... Good vendors are waiting five days to get paid and starting to lose patience. ... And the CFO has no clean view of what’s going out the door or why. ... So it’s not just a cost line — it’s a source of friction for basically everyone involved.”

“So that’s the problem. ... Manual, slow, error-prone, and risky. ... And that’s the thing I was asked to fix.”

PART 3

The goals — and how I read them

~3 min

NOTE This is the 'I actually understood the assignment' part. Say it plainly.

“OK — so before I show you anything, let me tell you what the goals actually were, and how I read them, because that shaped every decision. ... The case asked for a **multi-agent system** — not a single script, but several cooperating agents — that automates the whole workflow: ... ingestion, validation, approval, and payment. ... It asked that it be a *working prototype*, something that actually runs, not slides or a diagram. ... It asked for real agent behaviour — **tool use**, **structured outputs**, and a **self-correction loop**. ... And it said to use an LLM as the reasoning engine, and to simulate the external parts — the inventory and the payments — locally.”

“Now — here’s what I *understood* by all of that. ... To me, the point wasn’t ‘call an LLM and hope.’ ... The point was: build something that *behaves like a small finance team*, where each step is responsible for one job, where the system can catch its own mistakes, and where — crucially — you can **see and trust** every decision, because we’re talking about *moving money*. ... The case even planted deliberately broken and fraudulent invoices in the sample data. ... So I read that as a clear signal: ... the real test isn’t the happy path. ... It’s how gracefully the system handles the mess.”

“So that was my lens the whole way through: ... make the happy path work, but design for the mess, and make every decision explainable. ... OK — so based on that understanding, let me tell you what I actually built.”

PART 4

My approach — the shape of the solution

~2–3 min

POINT Turn to the screen now. Point to the header banner.

“So here it is. ... I called it GalatIQ. ... At the top level, it’s **four agents** that work like a small accounts-payable team, and an invoice flows through them left to right.”

POINT Open the ‘How this works’ panel and point along the four steps.

“**Ingestion** reads the invoice. ... **Validation** checks it against inventory and looks for fraud. ... **Approval** makes the approval call. ... And **Payment** either pays it or blocks it. ... Each of those is a separate agent with one job.”

“And here’s the first ‘why’. ... OK so **why four agents instead of one big prompt?** ... Two reasons. ... First, because we’re moving money, I wanted every step to be *inspectable and testable on its own*. If validation is wrong, I can test just validation. ... Second, it mirrors how a real AP team is actually organised — one person extracts, one checks, a VP signs off, finance pays. ... So the design maps onto the business, which makes it easy to explain and easy to trust.”

“Underneath, the four are wired together as a small **state graph** — the same idea as a framework like LangGraph, with a shared record that flows through named steps and branches at the end. ... OK so here I made a deliberate choice: I wrote that graph myself, in a small amount of code, instead of pulling in a heavy framework. ... Why? ... Because I wanted it lightweight, transparent, and dead easy for you to run. ... And if we ever wanted to scale it, moving to the real framework is mechanical, because the shapes already match.”

Controls

Sample invoice

invoice_1001.txt

...or upload your own

Upload

200MB per file • TXT, JSON, ...

Process invoice

Process all samples

What do these do?

Deploy

Galatix : Invoice Processing

Read any-format invoice, check it against inventory, approve it, and pay it — with every decision recorded.

How this works

Each invoice goes through four stages:

1. **Ingestion**: reads the file (TXT / JSON / CSV / XML / PDF) and pulls out the vendor, amount, line items and due date — fixing typos and scan noise, and scoring how confident it is in the data.
2. **Validation**: checks each item against the inventory database, flags stock shortfalls and unknown items, and scores risk from signals like a zero-stock item or urgent "wire transfer" wording.
3. **Approval**: applies the approval rules (invoices ≥ \$10,000 get extra review), then a second review pass double-checks the decision and can revise it.
4. **Payment**: approved invoices are paid; the rest are held for review or rejected.

Outcomes: **PAID** · **ESCALATED** (flagged for review) · **REJECTED** (fraud or bad data).

👉 Pick a sample invoice and hit **Process invoice**, or **Process all samples** for the overview.

The landing screen you're pointing at.

PART 5

Agent one — Ingestion (reading the mess)

~4–5 min

“Let me take the four agents one at a time, because each one has a story. ... First, **Ingestion** — the reader. ... Its job sounds simple: pull out the vendor, the amount, the line items, and the due date. ... But remember, the invoices are a mess, in five different formats — text, JSON, CSV, XML, and PDF.”

“OK so here I did a few specific things, and each one is because of something real in the data. ... The sample set has an invoice where the word ‘invoice’ is literally misspelt as I-N-V-O-C-E, and ‘vendor’ is written ‘V-N-D-R’. ... So I built the extractor to be tolerant of

that — it doesn't need the labels to be perfect. ... There's a scanned one where the year '2026' has a capital letter O instead of a zero, from bad OCR. ... So I added a repair step that fixes those number-versus-letter mix-ups — but only inside numbers, so it doesn't go and corrupt a vendor's name."

"And here's a design decision I'm happy with. ... Instead of the reader just failing when something's missing, it does two things. ... One, it gives itself a **confidence score** — 'how sure am I about what I just read?' ... And two, it tries to **repair** what it safely can — for instance, if the total is missing but the line items are there, it recomputes the total rather than giving up. ... If it genuinely can't be sure, it doesn't guess — it flags low confidence and sends the invoice to a person. ... OK so the principle there is: *fail safe, not silent*. ... I never want it to quietly invent a number on an invoice."

"And just to be clear on why the five formats matter — they're not just different file extensions, they're genuinely different *shapes*. ... A JSON invoice is neatly structured. ... A CSV might be a tidy table, or it might be a weird vertical key-value layout. ... An XML one nests everything in tags. ... And the text and PDF ones are basically free-form — an email with the invoice pasted in the body, or a scanned table with columns that don't quite line up. ... So the reader has to normalise all of those into one clean, consistent record before anyone downstream can trust it. ... OK so that normalisation is quietly one of the harder parts of the whole system, and it's where a lot of the real-world value is."

CLICK Pick a messy sample (e.g. *invoice_1012* or *invoice_1002*) and click Process. Point to the extracted fields.

"So watch — this is one of the messy ones. ... And you can see, despite the typos and the noise, it's pulled out clean, structured fields. ... That's the ingestion agent doing its job."

PART 6

Agent two — Validation (the guard)

~4–5 min

"Second agent — **Validation**. ... If ingestion is the reader, this one is the guard. ... It takes the structured data and checks it against a mock inventory database — I used SQLite, a real database that runs from a single file, so the whole thing stays self-contained."

“OK so what does it actually check? ... Three buckets, and I want to explain why I split them into three, because this is one of my favourite decisions. ... The first bucket is **hard errors** — things that block a payment outright: a negative quantity, a missing vendor, an item that’s in our database but has *zero stock* — which is a classic fake line. ... The second bucket is **review flags** — things that aren’t necessarily wrong, but a person should look: the order is bigger than our stock, or it’s an item we’ve never heard of, or it’s in a foreign currency. ... And the third bucket is **soft warnings** — little notes, like a price that’s a bit off catalogue, that we record but don’t act on.”

“And then, separately, there’s **fraud**. ... Now, the sample data deliberately includes fraudulent invoices, so I took that seriously. ... OK so here I did this: instead of one fraud rule, I built a *fraud score* that adds up independent signals. ... A zero-stock item is worth some points. ... High-pressure language — ‘pay immediately, wire transfer preferred’ — worth some points. ... A due date that says ‘yesterday’ — worth some points. ... The idea is, one weird thing on its own is just a warning ... but three of them stacking up is a rejection. ... That’s much harder to fool than a single yes-or-no rule, and it mirrors how a suspicious human actually thinks — it’s the *combination* that feels wrong.”

“There’s also a quiet bit of maths in here that I think matters. ... The agent doesn’t just trust the totals on the invoice — it re-checks them. ... Do the line items actually add up to the subtotal? ... Does subtotal plus tax equal the stated total? ... And if the same item appears on several lines — which happens on the bulk orders — it *adds those quantities together* before checking stock, so someone can’t sneak past the limit by splitting one big order across five lines. ... OK so I did that because catching the obvious fraud is easy; it’s the quiet inconsistencies that actually cost companies money.”

CLICK Pick invoice_1003 (the \$100K fraud) and click Process. Let REJECTED land, then point to the fraud signals.

“So here’s the one the system should hate. ... A hundred-thousand-dollar invoice. ... And — rejected. ... Look why: ... zero-stock item, urgent wire-transfer language, and a due date of ‘yesterday.’ ... Three signals, a high score, blocked. ... Nobody had to catch that manually. ... *That* is the two-million-dollar leak, caught automatically.”

PART 7

Agent three — Approval, and the self-correction loop

~4 min

“Third agent — **Approval**. ... This is the brain, and it’s where the LLM really earns its place, because this is the step that needs *judgment*, not just rules. ... It plays the role of the approver. ... There’s a rule I built in: anything **ten thousand dollars or more** gets extra scrutiny.”

“But here’s the part I’m proud of, and it’s a self-correction step I built in. ... OK so here I did this: the approval agent doesn’t just make one call and move on. ... It makes a first decision ... and then it runs a **self-review pass** — a second pass where it *critiques its own decision* before committing. ... And that loop can genuinely change its mind — in both directions. ... It can catch a case where it was about to approve something that still had a blocker ... and it can also relax a case where it was being too harsh.”

CLICK Pick invoice_1017 and click Process. Point to the ‘self-corrected’ line.

“Let me show you exactly that. ... This invoice is twelve-and-a-half thousand — so it’s over the scrutiny line, and the first decision is ‘escalate, send it for review.’ ... But then the self-review step looks again and goes: ... ‘hang on — it’s big, yes, but it’s completely clean. Everything’s in stock, the maths is right, there’s no fraud.’ ... So it **overrules its own first call** and approves it. ... And you can see that right here — escalate, then approve, after the self-review. ... OK so that, to me, is the difference between something that follows rules blindly ... and something that actually reasons about what it’s doing.”

PART 8

Agent four — Payment (the safe hands)

~1–2 min

“And the fourth agent — **Payment** — is the simplest, on purpose. ... It’s the only place money can move, so I kept it dead simple and predictable. ... If the invoice is approved, it calls the payment API — simulated here — and records a reference. ... If it was flagged, it holds it for review. ... If it was rejected, it logs the reason and pays nothing. ... One place, one responsibility, no surprises. ... OK so the thinking there is: keep the money-moving step boring and auditable.”

PART 9

The whole batch — the business view

~3 min

CLICK Click 'Process all samples'. Let the numbers and chart draw.

"So that's one invoice at a time. ... But let me zoom out, because this is the view a finance leader actually cares about. ... If I run the whole batch ... here's the scorecard. ... Out of seventeen invoices: **eight paid automatically**, ... **seven sent for review**, ... and **two rejected** outright."

POINT Point to the 'held from auto-pay' number.

"And this number here is the one I'd put in front of a CFO. ... It's the total dollar value the system **stopped** from being paid automatically without a second look. ... That's the risk it's catching, in money terms. ... The clean ones flow straight through in seconds, and the humans only spend their time on the handful that actually need a person."

CLICK Click 'View rejected', then open one of the two.

"And it's not a static report — it's interactive. ... If I click 'rejected' ... it pulls up exactly the two that were blocked ... and I can open either one and see the full reasoning behind it. ... So a reviewer goes from the big number straight to the specific invoice in two clicks. ... OK so I built it that way because a dashboard nobody can drill into is just a poster — I wanted this to be something a team would actually use."

PART 10

The decisions and trade-offs

~3–4 min

NOTE You can step back from the screen and just talk. This is the part interviewers remember.

"Let me pull back and talk about the decisions, because I think the reasoning matters more than the features. ... OK so, a few big ones."

"**First — three outcomes, not two.** ... Pay, flag for review, reject. ... I could've done a simple yes-or-no, but in real accounts-payable most problems aren't fraud — they're a typo or an item a buyer needs to confirm. ... Rejecting those outright would burn good vendors. ... So I flag them for review and save a hard reject for genuine fraud and bad data. ... That one decision is what makes it feel like a real AP tool instead of a toy."

“Second — I designed for resilience. ... The judgment runs on the LLM, but the hard guardrails — stock, thresholds, fraud rules — are fixed code around it. ... So if the model is ever slow or unavailable, the system still makes a safe, sensible call instead of stopping a payment run. ... In finance, ‘never blocks, never pays blindly’ beats ‘clever but fragile’ every time.”

“Third — I was honest about scope. ... This is a prototype, and part of the skill the case is testing is knowing what to cut. ... So I deliberately left out real image-OCR — I read text-based PDFs — and live currency conversion — I flag foreign-currency invoices for review instead of guessing an exchange rate. ... I’d rather ship a solid, honest core than a wide, shaky one.”

“Fourth — the interface is for a team, not a demo. ... I deliberately made the dashboard interactive — you click an outcome and drill straight into those invoices, and open any one to see the full reasoning. ... OK so I did that because a dashboard nobody can dig into is just a static poster, and the people who’d actually use this — a clerk, a VP — need to go from a number to a specific invoice in a click or two.”

“And under all of it — **observability**. ... Every decision is on screen with its reasoning, and every run is recorded. ... Because for something that moves money, ‘trust me, it works’ isn’t good enough. ... You have to be able to see *why*. ... That, honestly, was my north star the whole way through — make it work, make it safe, and make it *explainable*.”

PART 11

What I’d do next

~1–2 min

“If this were going to production, the path is pretty clear. ... I’d add real OCR for scanned images. ... I’d wire in live currency conversion so foreign invoices flow instead of pausing. ... I’d put a proper queue and database behind it for volume, and connect the real payment and inventory systems in place of the simulated ones. ... The nice thing is, none of that is a rewrite — the architecture already has a slot for each of them. ... OK so that was intentional: build the MVP so the next steps just plug in.”

PART 12

Setup, if anyone wants to run it

~1 min

NOTE *Only say this if they ask how to run it, or you're handing it over. Otherwise skip to the close.*

“And if you want to run it yourselves, it’s genuinely two steps plus a key. ... Install the requirements, build the small database with one command, and run the dashboard. ... The only thing you add is a Gemini API key — it goes in one line in the config file, line nine, or as an environment variable. ... There’s a short setup sheet in the repo that spells it out. ... After that, it just runs.”

PART 13

Closing

~1 min

PAUSE *Pause. Look up from the screen. Land it calmly, then stop talking.*

“So — that’s Galatiq. ... I took a slow, manual, risky process, and I turned it into four cooperating agents that read any format, check every invoice, reason about it, and pay or block it — in seconds, and with every decision visible. ... It catches the fraud, it clears the clean ones automatically, and it only puts a person on the handful that genuinely need one. ... I’m happy to go deeper on any part, or open up the code and walk through it. ... Thank you.”

Likely questions, and how to answer them

“Why not just make one big LLM call do the whole thing?”

“A single call is a black box — hard to test, and hard to trust with real money. Splitting it into agents means every step is inspectable and unit-tested, and I control exactly where money is allowed to move. It also mirrors a real AP team, which makes it explainable to the business.”

“What happens if the LLM gets it wrong, or is down?”

“The hard rules — stock, thresholds, fraud signals — are deterministic code, not the model. The LLM only handles judgment and phrasing; the guardrails are fixed. And if a model call fails, it falls back to those rules, so a payment run never breaks on an API hiccup.”

“How does it handle a completely new invoice format?”

“The parser already handles the five common ones and normalises messy text. For a brand-new layout, that’s exactly where an LLM extraction step slots in — the interface for it is already there, so it’s an addition, not a rewrite.”

“How is the fraud detection actually working?”

“It’s a score, not a single rule. Independent signals — a zero-stock item, urgent wire-transfer language, an impossible due date — each add points. One is a warning; several stacking together tips it into a rejection. It’s the combination that flags fraud, which is much harder to game.”

“Is this production-ready?”

“It’s a solid MVP with tests and full observability. For production I’d add OCR, currency conversion, a durable queue, and the real payment and inventory integrations — but the architecture is built to take them without a rewrite.”

“How do you know it actually works?”

“Thirty-one automated tests, including an end-to-end check that all seventeen sample invoices reach the decision I expect. So if I change something and break a case, I know immediately.”

“Why did you choose Gemini?”

“Mostly interchangeable for this — the engine is provider-agnostic behind one setting, so Gemini, Grok, OpenAI or Anthropic all drop in by changing one line. Gemini gives a strong, fast model with a generous free tier, which suits a prototype.”

“Why SQLite for the inventory?”

“It’s a real database with zero setup — one file, no server — so the whole thing stays self-contained and portable. Swapping it for the company’s real inventory system is just changing one query.”